

Apache Airflow

Local Executor & Parallel Task Execution

Hands-On Implementation Lab Guide

Configure · Execute · Monitor · Optimize

6 Labs Step-by-Step	LocalExecutor Deep Dive	Parallel Tasks Hands-On	Scheduler Tuning Production Patterns
-------------------------------	-----------------------------------	-----------------------------------	--

Prerequisites

AWS EC2 Ubuntu VM • Docker + Docker Compose installed • Airflow deployed via Docker • VM stopped and needs restart

Table of Contents

Section 1 Environment Setup — Restarting Your EC2 & Airflow

- › 1.1 Start the EC2 Instance
- › 1.2 Reconnect via SSH
- › 1.3 Start Docker Compose Stack
- › 1.4 Verify All Services

Section 2 Understand the Current Executor

- › 2.1 Check Default Executor
- › 2.2 Executor Architecture Comparison

Section 3 Configure LocalExecutor

- › 3.1 Edit docker-compose.yaml
- › 3.2 Modify airflow.cfg / Environment Variables
- › 3.3 Restart Stack & Verify

Section 4 Execute Parallel Tasks — Use Case Lab

- › 4.1 Create a Parallel ETL Pipeline DAG
- › 4.2 Trigger and Monitor Execution
- › 4.3 Verify Parallelism in UI

Section 5 Analyze Resource Usage

- › 5.1 Monitor CPU & Memory
- › 5.2 Docker Stats
- › 5.3 Airflow Task Metrics

Section 6 Modify Airflow Configuration for Throughput

- › 6.1 Key Configuration Parameters
- › 6.2 Apply Changes

Section 7 Test Scheduler Throughput & High-Volume Simulation

- › 7.1 Bulk DAG Generator Script
- › 7.2 Scheduler Stress Test
- › 7.3 Observe & Tune

Appendix Troubleshooting & Quick Reference

1

Environment Setup — Restarting Your EC2 & Airflow

Start your stopped VM and bring up the full Docker stack

■ Section Objective

- Your EC2 instance was stopped to save costs. You need to restart it, reconnect via SSH, and bring the Airflow Docker Compose stack back online.
- By the end of this section all Airflow services (webserver, scheduler, worker, postgres, redis) will be running and healthy.

LAB 1.1 | Start the EC2 Instance (AWS Console)

Step-by-Step: Start Instance from AWS Console

1 Login to AWS Console

Open <https://console.aws.amazon.com> and sign in with your credentials.

2 Navigate to EC2 Instances

Click Services → EC2 → Instances (Running). Change the filter to show Stopped instances.

3 Select Your Instance

Tick the checkbox next to your Airflow instance (look for the Name tag you used during setup).

4 Start the Instance

Click Instance State → Start Instance. Wait for Instance State to turn green (running) — takes 30-60 seconds.

5 Copy Public IP

Once running, copy the new Public IPv4 address shown in the Details tab. Note: the IP changes every time you restart unless you use an Elastic IP.

■ Important — IP Address Changes on Restart

- AWS assigns a NEW public IP every time a stopped instance restarts (unless Elastic IP is attached).
- Always copy the new IP from the EC2 console before trying to SSH.

LAB 1.2 | Reconnect via SSH

Open a terminal on your local machine (Git Bash / PuTTY on Windows, Terminal on Mac/Linux):

SSH Connection

```
# Replace <NEW_PUBLIC_IP> with the IP from EC2 console
# Replace ~/Downloads/your-key.pem with your actual .pem key path
```

```
ssh -i ~/Downloads/your-key.pem ubuntu@<NEW_PUBLIC_IP>

# If you get a permission warning on first use, type: yes
# If permission denied error, fix key permissions:
chmod 400 ~/Downloads/your-key.pem
```

Once connected, your prompt will look like:

Expected Prompt

```
ubuntu@ip-172-31-xx-xx:~$
```

LAB 1.3 | Start the Docker Compose Stack

Navigate to your Airflow project directory and start all containers:

Start Stack

```
# Navigate to your Airflow docker directory
cd ~/airflow-docker # or wherever your docker-compose.yaml is

# Check current container status
docker ps -a

# Start all services in detached mode
docker compose up -d

# Watch the startup logs (Ctrl+C to exit watching)
docker compose logs -f
```

✓ Startup Takes ~60-90 Seconds

- Postgres initializes first, then Redis, then the scheduler and webserver.
- You may see 'health checks failing' initially — this is normal. Wait for all containers to show 'healthy'.
- Run 'docker compose logs scheduler' separately if you see errors.

LAB 1.4 | Verify All Services Are Healthy

Verify Status

```
# Check all containers are running
docker ps

# Expected output – all STATUS should show (healthy) or Up
# CONTAINER ID IMAGE STATUS PORTS
# xxxx airflow-webserver Up 2 min (healthy) 0.0.0.0:8080->8080/tcp
# xxxx airflow-scheduler Up 2 min
# xxxx airflow-worker Up 2 min
# xxxx postgres Up 2 min (healthy) 5432/tcp
# xxxx redis Up 2 min (healthy) 6379/tcp
```

Access the Airflow UI in your browser:

Access UI

```
# Open in browser:
http://<NEW_PUBLIC_IP>:8080

# Default credentials:
Username: airflow
Password: airflow
```

Command	Purpose
docker compose up -d	Start all services in background
docker compose down	Stop and remove all containers
docker compose restart	Restart all containers
docker compose logs -f [service]	Follow logs for a specific service
docker ps	List running containers with status
docker exec -it bash	Open a shell inside a container

2

Understand the Current Executor

Learn what executor is running and why it matters

■ Section Objective

- Identify which executor Airflow is currently using.
- Understand the difference between SequentialExecutor (default) and LocalExecutor.
- Learn when each executor is appropriate.

LAB 2.1 | Check the Current Executor

Check Executor

```
# Shell into the scheduler container
docker exec -it airflow-airflow-scheduler-1 bash

# Inside the container, check the config
airflow config get-value core executor

# OR check environment variable
echo $AIRFLOW__CORE__EXECUTOR

# Exit the container
exit
```

Alternatively, from the Airflow UI: Admin → Configuration → search for 'executor'

Executor Comparison

Executor	Description	Parallelism	Use Case
SequentialExecutor	Runs ONE task at a time, no parallelism. Uses SQLite. Default for quick testing only.	Single process	Development only
LocalExecutor	Runs tasks in PARALLEL using Python subprocess. Uses PostgreSQL. Best for single-VM setups.	Multi-process	Dev + Small Prod
CeleryExecutor	Distributes tasks across multiple worker nodes via a message queue (Redis/RabbitMQ).	Multi-node	Large Scale Prod

KubernetesExecutor	Runs each task in its own Kubernetes Pod. Maximum isolation.	K8s Pods	Enterprise Cloud
--------------------	--	----------	------------------

■ Why LocalExecutor for This Lab?

- You have a single EC2 VM — no Kubernetes cluster, no separate worker nodes.
- LocalExecutor gives you real parallelism on a single machine without extra infrastructure.
- It is the recommended executor for development and small production workloads.

3

Configure LocalExecutor

Switch from SequentialExecutor to LocalExecutor

■ Section Objective

- Edit the docker-compose.yaml to set AIRFLOW__CORE__EXECUTOR=LocalExecutor.
- Ensure PostgreSQL backend is configured (required — LocalExecutor does NOT work with SQLite).
- Restart the stack and confirm the executor change.

LAB 3.1 | Locate and Inspect docker-compose.yaml

Inspect Config

```
# On your EC2, navigate to your Airflow directory
cd ~/airflow-docker

# View the compose file
cat docker-compose.yaml | grep -A 3 'EXECUTOR'

# Also check the x-airflow-common section which holds shared env vars
grep -n 'EXECUTOR\|DATABASE\|SQL_ALCHEMY' docker-compose.yaml
```

LAB 3.2 | Edit docker-compose.yaml

Open the file with nano (or vi):

Open Editor

```
nano docker-compose.yaml
```

Find the **x-airflow-common** section (shared environment block). Locate the EXECUTOR line and change it:

docker-compose.yaml Changes

```
# BEFORE (default):
AIRFLOW__CORE__EXECUTOR: CeleryExecutor

# OR
AIRFLOW__CORE__EXECUTOR: SequentialExecutor

# CHANGE TO:
AIRFLOW__CORE__EXECUTOR: LocalExecutor

# Also ensure these are present (PostgreSQL is required):
AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: postgresql+psycopg2://airflow:airflow@postgres/airflow
AIRFLOW__CORE__LOAD_EXAMPLES: 'false'
AIRFLOW__CORE__PARALLELISM: '32'
AIRFLOW__CORE__MAX_ACTIVE_TASKS_PER_DAG: '16'
AIRFLOW__CORE__MAX_ACTIVE_RUNS_PER_DAG: '4'
```


Save and exit nano: Press **Ctrl+O**, then Enter, then **Ctrl+X**

■ LocalExecutor Requires PostgreSQL — Not SQLite

- If `AIRFLOW__DATABASE__SQL_ALCHEMY_CONN` points to `sqlite`, `LocalExecutor` will FAIL with a database locking error.
- The standard `docker-compose` from Apache Airflow already uses PostgreSQL. Verify the connection string above is present.

LAB 3.3 | Remove Worker Service (Optional but Recommended)

With `LocalExecutor`, you do NOT need a separate Celery worker container. You can comment it out to save memory:

Remove Worker

```
# In docker-compose.yaml, find the 'airflow-worker' service
# Comment out or remove the entire block starting with:
# airflow-worker:
# <<: *airflow-common
# command: celery worker
# ...

# Also remove redis dependency if no longer needed:
# redis:
# image: redis:7.2-bookworm
# ...
```

LAB 3.4 | Restart the Stack and Verify

Restart and Verify

```
# Stop all containers
docker compose down

# Start fresh with new config
docker compose up -d

# Wait ~60 seconds, then verify executor
docker exec -it airflow-airflow-scheduler-1 bash -c \
'airflow config get-value core executor'

# Expected output:
# LocalExecutor

# Also verify from UI:
# Admin → Configuration → search 'executor'
```

✓ Verify in the Airflow UI

- Go to <http://localhost:8080> → Admin → Configuration.
- Search for 'executor' — you should see LocalExecutor listed.
- The scheduler log will also print: 'Using executor LocalExecutor'.

4

Execute Parallel Tasks — Use Case Lab

Build and run a real parallel ETL pipeline DAG

■ Use Case: Parallel ETL Pipeline

- Business Scenario: An e-commerce company needs to extract data from 5 regional databases simultaneously, transform each region's data, and load to a central warehouse.
- Without parallelism: tasks run sequentially — slow and inefficient.
- With LocalExecutor: all 5 extract tasks run at the SAME TIME.

LAB 4.1 | Create the DAG File

On your EC2, navigate to the DAGs folder and create a new file:

Create DAG File

```
# Find your DAGs folder (check docker-compose volumes)
grep -i 'dags' docker-compose.yaml | grep volume

# Typically:
cd ~/airflow-docker/dags

# Create the DAG file
nano parallel_etl_pipeline.py
```

Paste the following complete DAG code:

```
parallel_etl_pipeline.py

from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.operators.empty import EmptyOperator
from datetime import datetime, timedelta
import time, random, logging

default_args = {
    'owner': 'data-team',
    'retries': 1,
    'retry_delay': timedelta(minutes=2),
    'email_on_failure': False,
}

REGIONS = ['north', 'south', 'east', 'west', 'central']

def extract_regional_data(region: str, **context):
    """Simulate extracting data from a regional database."""
    logging.info(f'[EXTRACT] Starting extraction for region: {region}')
    # Simulate DB query time (2-4 seconds per region)
    sleep_time = random.uniform(2, 4)
```

```
time.sleep(sleep_time)
record_count = random.randint(500, 5000)
logging.info(f'[EXTRACT] {region}: extracted {record_count} records in {sleep_time:.1f}s')
# Push result to XCom for downstream tasks
context['ti'].xcom_push(key=f'{region}_count', value=record_count)
return record_count

def transform_data(region: str, **context):
    """Simulate data transformation for a region."""
    count = context['ti'].xcom_pull(key=f'{region}_count',
    task_ids=f'extract_{region}')
    logging.info(f'[TRANSFORM] Processing {count} records for {region}')
    time.sleep(random.uniform(1, 2))
    transformed = int(count * 0.95) # 5% filtered out
    logging.info(f'[TRANSFORM] {region}: {transformed} records after cleaning')
    return transformed

def load_to_warehouse(**context):
    """Load all regional data to central warehouse."""
    total = 0
    for region in REGIONS:
        count = context['ti'].xcom_pull(task_ids=f'transform_{region}')
        total += count or 0
    logging.info(f'[LOAD] Loaded {total} total records to warehouse')
    time.sleep(1)
    return total

with DAG(
    dag_id='parallel_etl_pipeline',
    default_args=default_args,
    description='Parallel ETL for 5 regional databases',
    schedule_interval='@daily',
    start_date=datetime(2024, 1, 1),
    catchup=False,
    max_active_runs=1,
    tags=['etl', 'parallel', 'localexecutor-demo'],
) as dag:
    start = EmptyOperator(task_id='pipeline_start')
    end = EmptyOperator(task_id='pipeline_end')

    load = PythonOperator(
        task_id='load_to_warehouse',
        python_callable=load_to_warehouse,
        provide_context=True,
    )
```

```

for region in REGIONS:
    extract = PythonOperator(
        task_id=f'extract_{region}',
        python_callable=extract_regional_data,
        op_kwargs={'region': region},
        provide_context=True,
    )
    transform = PythonOperator(
        task_id=f'transform_{region}',
        python_callable=transform_data,
        op_kwargs={'region': region},
        provide_context=True,
    )

# Chain: start → extract_X → transform_X → load → end
start >> extract >> transform >> load >> end

```

LAB 4.2 | Trigger and Monitor the DAG

1 Open the Airflow UI

Navigate to `http://:8080` in your browser.

2 Find Your DAG

Look for 'parallel_etl_pipeline' in the DAG list. Toggle it ON (the blue switch).

3 Trigger a Run

Click the Play (■) button → Trigger DAG. Confirm in the dialog.

4 Open Graph View

Click on the DAG name → Graph. Watch the task boxes turn yellow (running) and green (success).

5 Check Task Logs

Click on any running task box → Log. You will see the print statements from the Python function.

LAB 4.3 | Verify Parallelism

Open a second terminal on your EC2 while the DAG is running:

Verify Parallel Processes

```

# Watch running processes in real time
watch -n 1 'ps aux | grep airflow | grep -v grep | wc -l'

# OR list all airflow task runner processes
ps aux | grep 'airflow tasks run'

```

```
# You should see 5 separate processes – one per region!
# Example output:
# ubuntu 12345 python airflow tasks run parallel_etl_pipeline extract_north ...
# ubuntu 12346 python airflow tasks run parallel_etl_pipeline extract_south ...
# ubuntu 12347 python airflow tasks run parallel_etl_pipeline extract_east ...
# ubuntu 12348 python airflow tasks run parallel_etl_pipeline extract_west ...
# ubuntu 12349 python airflow tasks run parallel_etl_pipeline extract_central ...
```

✓ Key Observation

- All 5 extract tasks start at roughly the SAME TIME — this is parallel execution.
- With SequentialExecutor, they would run one after another — taking 5x longer.
- In the Grid view, you will see the task run times overlap on the timeline.

5

Analyze Resource Usage

Monitor CPU, memory, and Docker container stats

■ Section Objective

- Monitor CPU and memory while parallel tasks are running.
- Use Docker stats to see per-container resource consumption.
- Understand which Airflow components consume the most resources.

LAB 5.1 | Monitor System Resources with htop

System Monitor

```
# Install htop if not present
sudo apt-get install -y htop

# Launch htop (interactive process viewer)
htop

# While DAG is running, observe:
# - CPU bars at top (multiple cores if t3.medium or larger)
# - Memory usage bar
# - Python processes listed for each task

# Press F6 to sort by CPU%
# Press q to quit
```

Quick Checks

```
# Quick CPU snapshot without htop
top -bn1 | head -20

# Memory usage
free -h

# CPU cores available
nproc

# Detailed CPU info
lscpu | grep -E 'CPU\(s\)|Thread|Core'
```

LAB 5.2 | Docker Container Resource Stats

Docker Stats

```
# Live stats for all containers (refreshes every 2 seconds)
docker stats

# One-time snapshot (no interactive mode)
docker stats --no-stream
```

```
# Example output during parallel execution:
# NAME CPU % MEM USAGE / LIMIT NET I/O
# airflow-scheduler-1 45.2% 512MiB / 2GiB 10MB / 5MB
# airflow-webserver-1 2.1% 380MiB / 2GiB 1MB / 800KB
# postgres-1 3.4% 150MiB / 2GiB 2MB / 1.5MB
# redis-1 0.5% 30MiB / 2GiB 500KB / 400KB
```

■ Understanding the Stats Output

- CPU %: Percentage of total CPU. Can exceed 100% on multi-core machines.
- MEM USAGE / LIMIT: Current memory used vs the container limit set in compose.
- NET I/O: Network traffic in/out — high during task log shipping.
- The scheduler container handles running subprocesses, so its CPU spikes during parallel task execution.

LAB 5.3 | Airflow Task Metrics from the UI

1 Task Duration View

In the Airflow UI, click on your DAG → Task Duration tab. This shows a bar chart of how long each task took.

2 Gantt Chart

Click the Gantt tab. This shows overlapping bars — parallel tasks will show as simultaneous horizontal bars.

3 Landing Times

In Browse → Task Instances, you can filter by DAG and see start_date and end_date for every task instance.

4 Compare Sequential vs Parallel

Note the total DAG run time. With parallelism, 5 x 3s tasks finish in ~3s total instead of ~15s.

UI Location	What to Look For
Gantt Chart	Visual overlap of task bars = parallelism working correctly
Task Duration	Compare individual task times across runs
Task Instances	Browse → Task Instances — full audit trail
DAG Runs	Browse → DAG Runs — total pipeline execution time

Logs

Click task box → Log — see stdout from Python functions

6

Modify Airflow Configuration for Throughput

Tune key parameters to maximize scheduler performance

■ Section Objective

- Understand the key Airflow configuration parameters that control parallelism and throughput.
- Apply these changes via environment variables in docker-compose.yaml.
- Test the effect of each change.

Key Configuration Parameters

Parameter	Recommended Value	Description
AIRFLOW__CORE__PARALLELISM	32	Max tasks running across ALL DAGs at once. The global ceiling.
AIRFLOW__CORE__MAX_ACTIVE_TASKS_PER_DAG	16	Max concurrent tasks within a single DAG run.
AIRFLOW__CORE__MAX_ACTIVE_RUNS_PER_DAG	4	How many DAG runs of the same DAG can be active simultaneously.
AIRFLOW__SCHEDULER__MAX_DAGRUNS_TO_CREATE_PER_LOOP	10	Scheduler creates this many DAG runs per scheduling loop.
AIRFLOW__SCHEDULER__MAX_DAGRUNS_PER_LOOP_TO_SCHEDULE	20	DAG runs processed per scheduler heartbeat loop.
AIRFLOW__SCHEDULER__SCHEДУLER_HEARTBEAT_SEC	5	How often the scheduler wakes up to check for new tasks (seconds).
AIRFLOW__SCHEDULER__MIN_FILE_PROCESS_INTERVAL	30	Seconds between re-parsing each DAG file. Lower = faster refresh, higher CPU.
AIRFLOW__SCHEDULER__DAG_DIR_LIST_INTERVAL	30	How often to scan the DAGs folder for new/deleted files.

LAB 6.1 | Apply Configuration Changes

docker-compose.yaml Tuning

```
# Edit docker-compose.yaml
nano ~/airflow-docker/docker-compose.yaml

# In the x-airflow-common environment section, add/update:
environment:
&airflow-common-env
AIRFLOW__CORE__EXECUTOR: LocalExecutor
AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: postgresql+psycpg2://airflow:airflow@postgres/airflow
# Parallelism settings
AIRFLOW__CORE__PARALLELISM: '32'
AIRFLOW__CORE__MAX_ACTIVE_TASKS_PER_DAG: '16'
AIRFLOW__CORE__MAX_ACTIVE_RUNS_PER_DAG: '4'
# Scheduler throughput
AIRFLOW__SCHEDULER__SCHEDULER_HEARTBEAT_SEC: '5'
AIRFLOW__SCHEDULER__MAX_DAGRUNS_TO_CREATE_PER_LOOP: '10'
AIRFLOW__SCHEDULER__MAX_DAGRUNS_PER_LOOP_TO_SCHEDULE: '20'
AIRFLOW__SCHEDULER__MIN_FILE_PROCESS_INTERVAL: '30'
AIRFLOW__SCHEDULER__DAG_DIR_LIST_INTERVAL: '30'
# Disable examples
AIRFLOW__CORE__LOAD_EXAMPLES: 'false'
```

Apply and Verify

```
# Apply changes: restart the stack
docker compose down
docker compose up -d

# Verify settings inside the scheduler
docker exec -it airflow-airflow-scheduler-1 bash

# Inside container:
airflow config get-value core parallelism
airflow config get-value core max_active_tasks_per_dag
airflow config get-value scheduler scheduler_heartbeat_sec
exit
```

■ Resource Limits on t2.micro / t3.micro

- If your EC2 is a t2.micro (1 vCPU, 1GB RAM), keep PARALLELISM at 8 or lower.
- Setting parallelism too high on a small instance causes memory swapping and SLOWER performance.
- Recommended instance types for this lab: t3.medium (2 vCPU, 4GB) or larger.

7

Test Scheduler Throughput & Simulate High-Volume Scheduling

Generate bulk DAGs and stress-test the scheduler

■ Section Objective

- Create 20 DAGs programmatically to simulate a high-volume production environment.
- Trigger all DAGs simultaneously and observe scheduler behavior.
- Measure scheduler latency, task pickup time, and identify bottlenecks.

LAB 7.1 | Create a Bulk DAG Generator

Create a script that generates 20 lightweight DAGs — each with 3 parallel tasks:

Create Generator

```
# Create the generator file
```

```
nano ~/airflow-docker/dags/bulk_dag_generator.py
```

bulk_dag_generator.py

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime, timedelta
import time, random

def simulate_work(task_id: str, dag_id: str, **kwargs):
    """Simulate a lightweight task that runs for 2-5 seconds."""
    duration = random.uniform(2, 5)
    print(f'[{dag_id}] Task {task_id} running for {duration:.1f}s')
    time.sleep(duration)
    print(f'[{dag_id}] Task {task_id} COMPLETE')

# Generate 20 DAGs dynamically
for dag_index in range(1, 21):
    dag_id = f'stress_test_dag_{dag_index:02d}'

    with DAG(
        dag_id=dag_id,
        default_args={
            'owner': 'load-test',
            'retries': 0,
            'email_on_failure': False,
        },
        description=f'Load test DAG #{dag_index}',
        schedule_interval=None, # Manual trigger only
        start_date=datetime(2024, 1, 1),
        catchup=False,
```

```
tags=['stress-test', 'bulk'],
) as dag:

tasks = []
for task_index in range(1, 4): # 3 parallel tasks per DAG
    t = PythonOperator(
        task_id=f'task_{task_index}',
        python_callable=simulate_work,
        op_kwargs={'task_id': f'task_{task_index}', 'dag_id': dag_id},
        provide_context=True,
    )
    tasks.append(t)

# All 3 tasks run in PARALLEL (no dependencies)
# tasks[0] tasks[1] tasks[2] -- all independent

# Register in global namespace (required for Airflow to discover)
globals()[dag_id] = dag
```

LAB 7.2 | Trigger All 20 DAGs Simultaneously

Use the Airflow CLI via Docker to trigger all DAGs at once:

Trigger All DAGs

```
# Wait for Airflow to discover the new DAGs (30-60 seconds)
# Check the UI – you should see 20 new stress_test_dag_XX entries

# Trigger all 20 DAGs using a bash loop
for i in $(seq -w 01 20); do
    docker exec airflow-airflow-scheduler-1 \
    airflow dags trigger stress_test_dag_${i}
    echo "Triggered stress_test_dag_${i}"
done

# Monitor progress
watch -n 2 'docker exec airflow-airflow-scheduler-1 \
airflow tasks states-for-dag-run stress_test_dag_01 $(date +%Y-%m-%d) 2>/dev/null'
```

■ What to Observe in the UI

- Go to Browse → DAG Runs — you should see 20 active runs.
- The scheduler will process them in batches based on MAX_DAGRUNS_PER_LOOP_TO_SCHEDULE.
- Total concurrent tasks = 20 DAGs x 3 tasks = 60 potential tasks, limited by PARALLELISM=32.

LAB 7.3 | Measure Scheduler Throughput

Measure Throughput

```
# Check scheduler logs for timing information
docker compose logs scheduler --tail=100 | grep -E 'heartbeat|scheduled|queued'

# Count currently running task instances
docker exec airflow-airflow-scheduler-1 \
airflow tasks states-for-dag-run --help 2>/dev/null

# Check how many tasks are in each state
docker exec airflow-airflow-scheduler-1 bash -c \
"airflow db shell <<< \"SELECT state, COUNT(*) FROM task_instance
WHERE updated_at > NOW() - INTERVAL '10 minutes'
GROUP BY state;\""

# Monitor CPU during stress test
docker stats --no-stream
```

LAB 7.4 | Analyze and Tune Based on Results

Symptom	Recommended Action
Scheduler CPU > 90%	Increase SCHEDULER_HEARTBEAT_SEC to 10, reduce DAGs_TO_SCHEDULE per loop
Tasks queued but not running	Check PARALLELISM limit — may need to increase
Long task pickup delay	Reduce MIN_FILE_PROCESS_INTERVAL, check scheduler log for errors
Memory > 80% on VM	Reduce MAX_ACTIVE_TASKS_PER_DAG, or upgrade EC2 instance type
Database connection errors	Increase AIRFLOW__DATABASE__SQL_ALCHEMY_POOL_SIZE to 10
DAG not appearing in UI	Check syntax: docker exec scheduler airflow dags list-import-errors

✓ Clean Up After Stress Testing

- After the lab, delete the bulk DAGs to keep your environment clean.
- In UI: DAGs list → select all stress_test_* → Actions → Delete DAG.
- Or delete the file: rm ~/airflow-docker/dags/bulk_dag_generator.py

A

Appendix — Troubleshooting & Quick Reference

Common errors and their solutions

Common Issues and Fixes

✗ Webserver unreachable (port 8080)

- Check EC2 Security Group: Inbound rule for TCP port 8080 from 0.0.0.0/0 must exist.
- Run: `docker ps` — is the webserver container actually running?
- Run: `docker compose logs webserver` — look for startup errors.
- EC2 IP changes on restart — confirm you are using the new IP.

■ Containers exit immediately after start

- Run: `docker compose logs` — look for 'Error', 'Permission denied', 'Connection refused'.
- Check disk space: `df -h` — Airflow needs at least 2GB free.
- Check if port 8080 is already in use: `sudo lsof -i :8080`
- Reset: `docker compose down -v` (WARNING: deletes volumes/DB) then `docker compose up -d`

■ LocalExecutor not taking effect

- Ensure you restarted with `docker compose down && docker compose up -d` (not just restart).
- Verify env var: `docker exec scheduler env | grep EXECUTOR`
- Check that SQLite is NOT being used (will cause immediate failure with LocalExecutor).

■ Tasks stuck in queued state forever

- Check PARALLELISM — may already be at max concurrent tasks.
- Check scheduler health: `docker compose logs scheduler | tail -30`
- Verify no zombie processes: `ps aux | grep 'airflow tasks run'`
- Try: `docker compose restart scheduler`

■ DAG not appearing in UI after creating file

- Wait up to 60 seconds for the scheduler to parse new files.
- Check for syntax errors: `docker exec scheduler airflow dags list-import-errors`
- Verify file is in the correct DAGs folder with .py extension.
- Check file permissions: `ls -la ~/airflow-docker/dags/`

Quick Reference: Most Used Commands

Quick Reference Commands

[illegible]

Architecture Recap — LocalExecutor on Docker

Container	Role with LocalExecutor
airflow-webserver	Serves the Airflow UI on port 8080. Reads from PostgreSQL DB.
airflow-scheduler	Parses DAGs, creates task instances, runs tasks as subprocesses (LocalExecutor).
postgres	Metadata database storing DAG runs, task states, XComs, logs.
redis	Not required with LocalExecutor — can be removed from compose.
airflow-worker	Not required with LocalExecutor — used only for CeleryExecutor.